

Observability Fundamentals — Easy KCNA Notes

1. What is Observability?

Observability is the ability to understand what is happening inside a system by looking at the information it produces.

In simple words:

Observability helps us see and understand the health and behavior of our application.

Imagine your application is a car 🚗.

You cannot see inside the engine while driving, so you use:

- Speedometer
- Fuel gauge
- Warning lights
- Engine diagnostics

These tell you what is happening inside the car.

Similarly, observability gives us information about what's happening inside our applications and Kubernetes cluster.

2. Why do we need Observability?

Imagine your application suddenly becomes slow:

 User



Application 🐌

You need to find out:

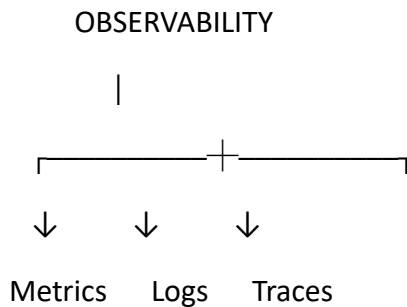
- Is the application using too much CPU?
- Is memory running out?
- Is a Pod failing?
- Is the network slow?

- Is a database request taking too long?
- Are there errors?

Observability helps you investigate these problems.

3. The 3 Pillars of Observability ★★

For KCNA, remember these **three important pillars**:



🧠 Easy memory:

Metrics = Numbers 📊

Logs = Events/Records 📄

Traces = Journey 🔍

4. Metrics 📊

What are Metrics?

Metrics are numerical measurements that tell us about the health and performance of a system.

Examples:

CPU Usage → 75%

Memory Usage → 60%

Requests → 1,000

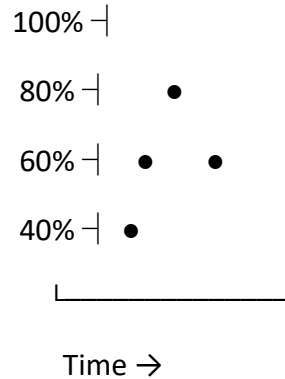
Error Rate → 5%

Latency → 200 ms

You can monitor these values over time.

Example:

CPU



If CPU suddenly reaches 100%, you know something may need investigation.

 **Remember:**

Metrics = What is happening? Measured in numbers.

5. Logs

What are Logs?

Logs are records of events that happen inside an application or system.

Example:

2026-08-07 10:20:01 User logged in

2026-08-07 10:20:05 Request received

2026-08-07 10:20:06 Database connected

2026-08-07 10:20:10 ERROR: Database connection failed

Logs can help you understand **what happened**.

 **Remember:**

Logs = Detailed records of events.

6. Traces 🔍

What is a Trace?

A **trace** follows a request as it travels through different services.

This is especially useful in **microservices**.

Example:

User



Frontend



User Service



Order Service



Payment Service



Database

A trace helps you follow this complete journey.

Example

Suppose a user clicks:

"Buy Now"

The request travels:

Buy Now



Frontend 10 ms



Order Service 20 ms



Payment 300 ms ← Slow!



Database 10 ms

The trace helps you identify that the **Payment Service** is taking most of the time.

 **Remember:**

Traces = Follow a request through the system.

7. Metrics vs Logs vs Traces

Type	Tells you	Example
Metrics 	What is happening?	CPU = 80%
Logs 	What happened?	Database connection failed
Traces 	Where did the request go? Frontend → Order → Payment	

Super easy trick:

 Metrics → NUMBERS

 Logs → EVENTS

 Traces → JOURNEY

8. Observability in Kubernetes

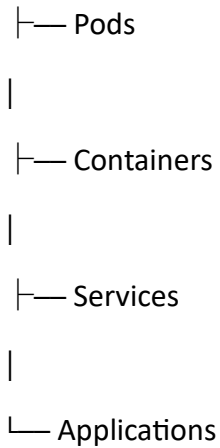
Kubernetes has many moving parts:

Cluster

|

|— Nodes

|



If something goes wrong, observability helps you understand the problem.

For example:

Pod



CPU = 95% 



Check Logs 



Find error



Check Trace 



Find slow service

9. Monitoring vs Observability ★

These terms are related but not exactly the same.

Monitoring

Monitoring generally tells you:

"Is something wrong?"

Example:

CPU > 90% 🚨

Observability

Observability helps you understand:

"Why is it wrong?"

Example:

CPU > 90%



Which Pod?



Which application?



Which request?



Which service caused it?

🧠 **Easy memory:**

Monitoring = Detect the problem

Observability = Understand/investigate the problem

10. Important Observability Questions

When troubleshooting an application, observability helps answer:

What happened?

📄 **Logs**

How much / how often?

📊 **Metrics**

Where did the request go?



11. Example: Website is Slow 🌐

Suppose users complain:

"The website is very slow!"

Step 1 — Metrics

Check:

CPU → 40%

Memory → 50%

Request latency → 2 seconds

You discover latency is high.

Step 2 — Logs

Check application logs:

ERROR: Database query timeout

Step 3 — Traces

Follow the request:

Frontend

↓ 20 ms

API

↓ 30 ms

Database

↓ 1950 ms ❌

Now you can see that the database call is causing the delay.

That's the power of observability.

12. Common Observability Tools

You may encounter tools such as:

 **Prometheus**

Commonly used for collecting and querying **metrics**.

 **Grafana**

Commonly used to **visualize metrics and create dashboards**.

 **Fluent Bit / Fluentd**

Commonly used for **collecting and forwarding logs**.

 **Jaeger**

Used for **distributed tracing**.

 **OpenTelemetry**

A widely used open-source framework for collecting and exporting **telemetry data**, including metrics, logs, and traces.

★ KCNA Revision Notes

Observability

- **Observability = understanding the internal state/behavior of a system from the data it produces.**
- Important for troubleshooting and understanding applications.
- Especially important in **microservices and Kubernetes**.
- Three major pillars:
 -  **Metrics**
 -  **Logs**
 -  **Traces**

Metrics

Numerical measurements.

Examples:

- CPU
- Memory
- Request count
- Error rate
- Latency

Logs

Records of events.

Examples:

- Errors
- Login events
- Application messages
- Database errors

Traces

Follow a request through multiple services.

User



Service A



Service B

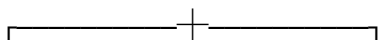


Service C



Final memory trick

OBSERVABILITY





Metrics



Logs



Traces



Numbers

Events

Journey

KCNA one-line answer:

Observability is the ability to understand and troubleshoot a system by analyzing its metrics, logs, and traces.